

Lab 10 Prompts: Implement SQLite CRUD and Orders Safely

Course: Create RESTful APIs and Web Apps with Python Flask Course code: TGS-2021010365 Version: v11.0
Maps to: A3, A4, A5

Prompt execution contract

Use `python ai_vibe.py --dry-run` to inspect the exact request. The script uses the OpenAI Python SDK Responses API with a Pydantic CodeBundle. It writes drafts only to `working-files/generated/`; it never overwrites trusted source. Keep `OPENAI_API_KEY` in the shell, not in prompts, files, screenshots or browser JavaScript.

Learner prompt - copy exactly for the first run

You are working on Lab 10 of a Northstar Commerce API. Implement parameterized SQLite repository functions for product/customer CRUD and atomic order creation. Validate stock inside the same transaction, insert order items, decrement inventory or roll back fully, use bounded pagination and an allow-list for PATCH columns. Add CRUD, insufficient-stock and injection-shaped-input tests.

Use only the synthetic context in `mock-data.json` and the current project files supplied by `ai_vibe.py`. Preserve existing API behaviour unless this prompt explicitly changes it. Use Python 3.11+, FastAPI, Pydantic v2, SQLite parameter binding and pytest/TestClient. Do not include secrets, database files, caches or invented test results. Return a typed CodeBundle containing complete file contents for only these targets:

- `app/repository.py`
- `tests/test_repository.py`

For every generated file, state its purpose. Include assumptions and concrete verification commands. The learner will review the diff and run the commands before accepting any code.

Evidence-led refinement prompt

The previous generated bundle did not yet pass the acceptance check below. Use the pasted failure evidence, identify the smallest contract or implementation mismatch, and return a revised CodeBundle containing only files that must change. Do not weaken or delete a passing test.

Acceptance check: Catalog/CRM CRUD and order tests pass, insufficient stock rolls back fully, missing IDs map to 404 and SQL values are parameter-bound.

Paste exact evidence here:

HTTP request/response, traceback, assertion diff or log request ID

Review checklist

- Confirm every generated path is inside `working-files/generated/`.
- Reject hard-coded keys, personal data, f-string SQL and unrestricted CORS.
- Compare generated behaviour with the written API contract and mock data.
- Run `pytest -q`; inspect status, JSON and SQLite state, not only the exit code.
- Record prompt, model name, assumptions, accepted diff and final evidence.